PATTERN RECOGNITION OF HANDWRITTEN DIGITS USING MNIST DATASET

[1]Meshaal.O Mouawad
Mississippi State University, Electrical and Computer Engineering Department
This technical report is to satisfy the requirement of Pattern Recognition course Spring, 2021
Final Project Report

***Abstract*—. MNIST, Modified National Institute of Standards and Technology, is the largest database of handwritten numbers used in deep learning, and machine learning. In this project, a hands-on experience of applying machine learning and pattern recognition techniques is given to a real-world data set such as MNIST. Multiple building blocks have been proposed and analyzed to improve the speed and the accuracy of the Convolutional Neural Networks (CNN). Two networks have been used with the same data. In network I, a three-layer MLP with ReLU and dropout resulting in fast training process with over all accuracy 95% during training and 94% for testing. Network II on the other hand, a stack of CNN, RelU, and Max pooling shows slower training process with better accuracy than network I and overall, 99% accuracy for training, and 98.9% for testing. Another modification on network II improved overall accuracy during the training to 99.82% and accuracy for testing to 99.25%. this modification will be shown in the report. The building blocks for the project will be discussed briefly with the results and figures. Python code is also provided for this project.**

***Index Terms:* Pattern Recognition, Convolutional Neural Networks, Deep learning, MNIST.**

I. INTRODUCTION

MNIST [1] stands for *Modified National Institute of Standards and Technology*. MNIST is a database of handwritten digits that used for training and testing many image processing systems and machine learning research. It has 60,000 training images and 10,000 testing images of handwritten digits and characters [2]. MNSIT database is good for people who want to try learning techniques and *pattern recognition* methods on real-world data while spending minimal efforts on preprocessing and formatting [1]. Later in 2017, an extended database has been published that has 240,000 training images and 40,000 testing images. MNIST is a standard training dataset for digits in English in recent times. Others provide similar databases for training digit datasets in other languages, and characters in English, and other languages. The dataset is considered as a benchmark for machine learning worldwide. It is perfect for those who want to explore machine learning methods and pattern recognition techniques. It reduces the time and effort that spend on preprocessing and formatting of data. an example of MNIST dataset is shown in Figure 1.

In this project, some neural layers have been used in the building blocks like *convolution layer*, *max pooling*, *flatten*, *dropout*, *dense*, and *activation layer*. Each layer has its duties to perform. They improve both the speed and accuracy of the *convolutional neural networks* (CNN). In this part, the layers used in this project would be explained shortly.

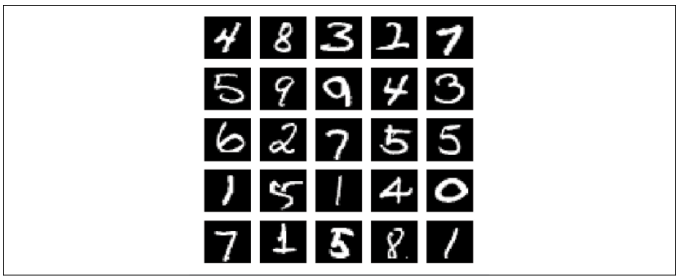


Figure 1: Example images from the MNIST dataset. Each grayscale image is 28 × 28-pixels. [9]

II. DEEP LEARNING ARTIFICIAL NEURAL NETWORKS

The four most common types of deep learning artificial neural networks layers are Fully connected, Convolution, Deconvolution, and Recurrent. In this project, the layers and the functions were used are convolutional layer, max pooling, flatten, dropout, dense, and fully connected layers.

A. Convolutional layer

A convolutional layer includes a series of filters that need to be taught about the parameters. The filter height and weight are less than the size of the input. To compute an activation map made of neurons, each filter is transformed with the volume of inputs. In computer vision, one of the difficulties is that images can be very large and thus computationally costly to work on. For practical uses, we need quicker and computationally cheaper algorithms. A simple neural network that is completely

[1]Meshaal Mouawad, Mississippi State University, Starkville, MS 39759 USA (e-mail: mm4922@msstate.edu).
This report was submitted for final project to satisfy the requirements of PhD level course to Dr. Tang Bo.

The information were used mostly from "Advanced Deep Learning with TensorFlow 2 and Keras - Second Edition," *Packt*. [Online]. Available: https://www.packtpub.com/product/advanced-deep-learning-with-tensorflow-2-and-keras-second-edition/9781838821654. [Accessed: 13-Apr-2021].

connected will not help.

B. *Max pooling*

Max pooling is a form of operation usually applied to individual convolution layers of CNNs. The key idea behind a layer of pooling is to "accumulate" characteristics from maps created by converting a filter over an image. Formally, its purpose is to gradually reduce the representation's spatial size to reduce the number of parameters and computation within the network. Max pooling is the most common method of pooling. Max pooling decreases the dimensionality of images when applied to a model by reducing the number of pixels in the output from the preceding convolution layer.

C. *Fully connected layers*

Fully connected layers connect every neuron in one layer to every neuron in the next layer. Fully connected layers are found in all different types of neural networks ranging from standard neural networks to convolutional neural networks (CNN) [7].

D. *Flatten*

Flatten is used to flatten the input. For example, if flatten is applied to a layer having input shape as (batch size, 1,1), then the output shape of the layer will be (batch size, 2). Flattening occurs when a reduction of all layers to one background layer is needed. Layers will increase the size of files, while also tying up available resources for processing. Anyone can choose to combine some layers or even flatten the whole image to one background layer to keep down the file size.

E. *Dense*

A regular deeply connected neural network layer is a dense layer. It is the most common layer, and it is sometimes used. The dense layer performs the input operation below and returns the output. The dense layer is also known as a fully connected layer. Fully connected or dense layers are those layers in a neural network where all inputs from one layer are connected to each activation unit of the next layer. The last few layers are fully connected layers in most common machine learning models that compile the data extracted from previous layers to form the final output.

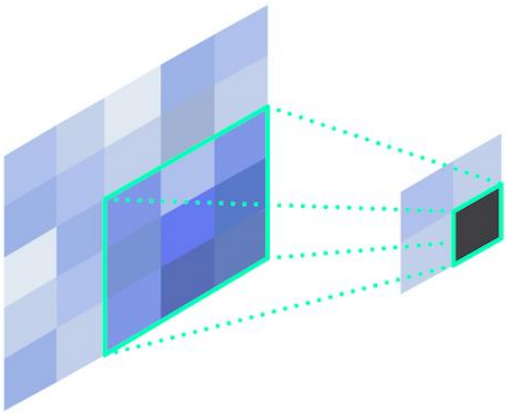


Fig. 1. Magnetization as a function of applied field. Note that “Fig.” is abbreviated. There is a period after the figure number, followed by two spaces. It is good practice to explain the significance of the figure in the caption.

F. *Dropout*

A single model can be used by randomly dropping out nodes during training to simulate having a large number of distinct network architectures. This is called *dropout* and provides a highly computationally cheap and surprisingly efficient method of regularization to minimize overfitting and increase generalization error in all forms of deep neural networks. The *Dropout layer* at random sets input units to 0 at each stage during the training period with a rated frequency that helps prevent overfitting. Inputs not set to 0 are scaled to 1/(1-rate) such that there is no difference in the sum for all inputs.

G. *Activation function*

In neural networks, a node's activation function determines the node's output given an input or set of inputs. Depending on the input, a typical integrated circuit is a digital network of activation functions that can be 'ON' (1) or 'OFF' (0). One linear function enables each layer. In turn, this activation goes as an input into the next step and the second layer calculates the weighted sum on that input and, in turn, fires based on another feature of linear activation.

III. The MNIST Digit Classifier Model

The proposed MLP model shown in Figure 1.3.3 can be used for MNIST digit classification. When the units or perceptron are exposed, the MLP model is a fully connected network, as shown in Figure 1.3.4. The output of the perceptron is computed from inputs as a function of weights, wi, and bias, bn, for the n-th unit. The corresponding tf.keras implementation is illustrated in Listing 1.3.2 [9]. The classifier model is implemented using the Sequential API of Keras in this project.

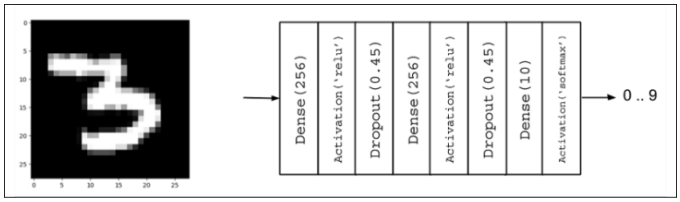


Figure 1.3.3: The MLP MNIST digit classifier model [9].

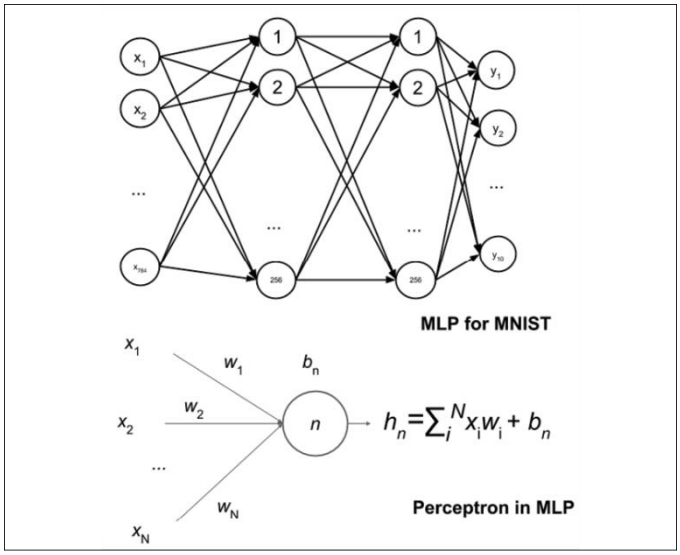


Figure 1.3.4: The MLP MNIST digit classifier in Figure 1.3.3 is made of fully connected layers. For simplicity, the activation and dropout layers are not shown. One unit or perceptron is also shown in detail [9].

IV. Approaching Methods

Keras deep learning library, a popular deep learning library with over 370,000 developers using it at the time of writing – a number that is increasing by about 35% every year [9] is implemented in this project. Google's *TensorFlow*, a popular opened source deep learning library, uses *Keras* as a high-level API for its library [9] Keras is used by Google, Netflix, Uber, and NVIDIA [9]. There are two networks were used in this project from opened source GitHub [8]. Since the dataset were already given the choice of the *loss function*, *the optimizer*, and *the regularizer*, should be determined that the model can be trained. For optimization, an ***Adaptive Moments (Adam)*** *algorithm* were used.

The MNIST dataset for hand-written digits is fed into two neural networks. Each network detail will be explained below:

1. Network I:

This network consists of a 3-layer multilayer perceptron (MLP) network where it has RelU activation and dropout after each layer. ReLU sets all non-positive values in the matrix to zero, and rest of the values are remained steady. Dropout reduces overfitting in neural networks and precludes complicated co-alterations on training data. The model is sequential. The batch size is set to 128, hidden_unit is set to 32, and dropout is 0.45 initially. The network is tested with changing different parameters. To get the output with a one-hot encoded vector, the SoftMax activation function is used. The SoftMax function uses an input vector and stabilizes it into a likelihood division. The network architecture is presented briefly in Figure 5.

The total number of parameters used for Network-1 = (25120 + 1056 + 330) = 26506.

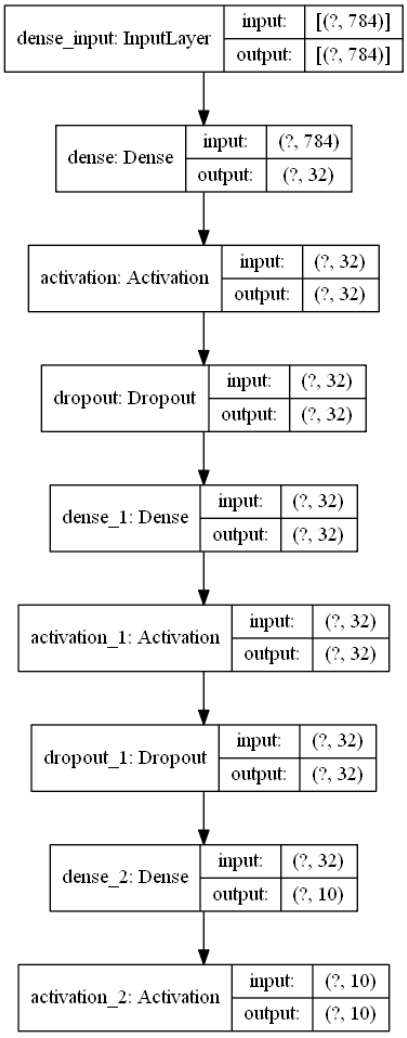


Fig. 5. Network architecture of network-1 of MNIST dataset (mp1_nn1.py)

TABLE I
THE COMPONENTS OF NETWORK 1

Layer Type	No. Parameters	Output Size
Dense	25120	32
Activation	0	32
Dropout	0	32
Dense 1	1056	32
Activation 1	0	32
Dropout 1	0	32
Dense 2	330	10
Activation 2	0	10
Total layers: 3		
Total Components:	26506	

Summery Summary of MNIST digit classifier model from Python

Total Model summary is illustrated in **Figure-2.** Total trained parameters for this network are 26506.

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 32)                25120
_________________________________________________________________
activation (Activation)      (None, 32)                0
_________________________________________________________________
dropout (Dropout)            (None, 32)                0
_________________________________________________________________
dense_1 (Dense)              (None, 32)                1056
_________________________________________________________________
activation_1 (Activation)    (None, 32)                0
_________________________________________________________________
dropout_1 (Dropout)          (None, 32)                0
_________________________________________________________________
dense_2 (Dense)              (None, 10)                330
_________________________________________________________________
activation_2 (Activation)    (None, 10)                0
=================================================================
Total params: 26,506
Trainable params: 26,506
Non-trainable params: 0
```

Figure-2: Model Summary of Network-1

In network 1, Adam optimizer is used. The loss function of this classifier is set as categorical cross-entropy. The model with 20 epochs was trained initially. In this network, the training process is much faster. It takes 1 second per epoch. In these network settings, the training accuracy of 95% and a test accuracy of 94%. The discuss of overall network accuracy in the latter part of this report.

2. Network II:

A stack of CNN-ReLU Maxpooling model is established to test the network for the same dataset. It is a combination of a 3-layer 2D convolutional layer. It has a 2D Maxpooling layer and a dropout layer. A dataset of 28 X 28 grayscale image is fed into the network. Initially, the batch size is set to 128. The kernel size is 3 and the pool size is 2. In the beginning, the filter size was 16, and the dropout was 0.3. The output layer is a 10-dimension one-hot vector, which can be done by using the SoftMax layer. Regularization is done by adding a dropout layer. The total network architecture is presented in Figure-3.

TABLE 2
THE COMPONENTS OF NETWORK II

Layer Type	No. Parameters	Output Size
Max_pooling2d	0	32
Conv2d_1	2320	32
Max_pooling_1	0	32
Conv2d_2	2320	32
Flatten	0	32
Dropout	0	10
Dense	1450	10
Activation	0	10
Total: 6		
Total	6250	

Summery Summary of MNIST digit classifier model from Python

The total number of parameters used for Network-2 = (160+2320+2320+1450) = 6250. Total Model summary is illustrated in **Figure-4.** Total trained parameters for this network is 6250.

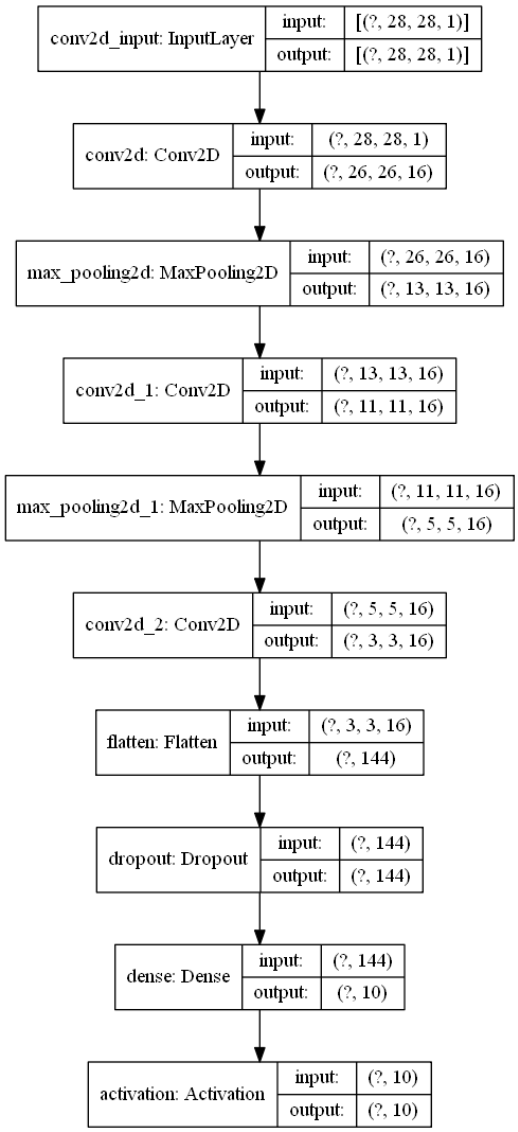


Figure-3: Network architecture of network-2 of MNIST dataset (mp1_nn2.py).

In this sequential neural network, the input size is 28 X 28 gray scale image (channel 1). This input is fed into a *conv2D* layer where is output size is 28 X 28.

Total number of parameters is used in this layer is K(LMN+1) = 16(3 X3 + 1) = 160

After 2D *MaxPooling*, the Kernel size become half, after the next dense layer,

The number of parameters used in the next dense layer = K(LMN+1) = 16(16 X 3 X3 + 1) =2320

Once again, after applying the maxpooling and conv2D, the total number of used parameters in this layer is K(LMN+1) = 16(16 X 3 X3 + 1) =2320

Lastly, after flattening, the size of the kernel become 144. After dropping out, the total = 144(1X 3 X3 + 1) =1450

The below table shows each component used in the network 2, their parameter number, and output size.

```
Loading MNIST dataset.
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 16)        160
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 13, 13, 16)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 11, 11, 16)        2320
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 5, 5, 16)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 3, 3, 16)          2320
_________________________________________________________________
flatten (Flatten)            (None, 144)               0
_________________________________________________________________
dropout (Dropout)            (None, 144)               0
_________________________________________________________________
dense (Dense)                (None, 10)                1450
_________________________________________________________________
activation (Activation)      (None, 10)                0
=================================================================
Total params: 6,250
Trainable params: 6,250
Non-trainable params: 0
```

Figure-4: Model Summary of Network-2

In network 2, '*Adam*' is used as an optimizer. The loss function of this classifier is set as categorical cross-entropy. The model was trained with 10 epochs initially. In this network, the training process is a little bit slower than the network 1. It takes an average of 11 seconds per epoch. In these network settings, the results of a training accuracy is 99.023% and a test accuracy of 98.9%. the discussion of overall network accuracy in the latter part of this report.

a modification of the network-2 was done to get better performance. The parameters were changed such as *batch size, kernel size, pool size, filter size,* and *dropout*. Moreover, the optimizer SGD (stochastic gradient descent) was changed instead of '*Adam'*. After several trial and error, results an accuracy of 99.82% during the training set and 99.25 for the testing set. Though it took near about 50 seconds per epoch during the training sessions, optimizer 'Adam' gave us a good result according to testing. The total parameter used for this training process is presented in Figure-5. The training

TABLE 2
THE TRAINING PARAMETERS MODIFICATION OF NETWORK II

Parameters	Changed Value
Batch size	64
Kernel size	3
Filters	64
Dropout	0.3

Summary of the modification of the network-2 for better performance

parameters that have been used for the modified network is illustrated in Table 3.

```
Loading MNIST dataset.
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 26, 26, 64)        640
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 13, 13, 64)        0
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 11, 11, 64)        36928
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 5, 5, 64)          0
_________________________________________________________________
conv2d_2 (Conv2D)            (None, 3, 3, 64)          36928
_________________________________________________________________
flatten (Flatten)            (None, 576)               0
_________________________________________________________________
dropout (Dropout)            (None, 576)               0
_________________________________________________________________
dense (Dense)                (None, 10)                5770
_________________________________________________________________
activation (Activation)      (None, 10)                0
=================================================================
Total params: 80,266
Trainable params: 80,266
Non-trainable params: 0
```

Figure-5: Model Summary of Modified Network-2

V. APPROACHING THE RESULTS & DISCUSSION

We have trained same dataset using different network settings. We will discuss our result for both networks below.

A. Network I

For the first network, we have trained the model using 10 epochs. The confusion Matrix for network-1 for both training and testing is illustrated below-

```
Train Confusion Matrix: (Rows are true classes, columns predictions)

  5840     0     7     0     4     4    26     2    36     4
     1  6598    29    23     5     3     1    13    57    12
    30    25  5667    44    32     3    30    49    74     4
     9     5   111  5727     2   104     7    54    74    38
     8    14     8     4  5560     0    72     3    27   146
    40     5    17    80    24  5078    59     1    80    37
    29    11     1     0    11    65  5787     1    13     0
    11    21    42    13    30     3     3  6025     9   108
    28    82    31    34    22    63    18     6  5510    57
    22     7     0    48   138    42     4    45    73  5570

Confusion matrix Train stats:
Total instances          : 60000
Overall accuracy    (OA)  : 0.95603


Test Confusion Matrix: (Rows are true classes, columns predictions)

   960     0     0     1     0     2    12     2     3     0
     0  1114     3     2     0     1     4     2     9     0
     4     5   977    10     5     1     6    11    12     1
     0     1    13   950     0    18     0    11    12     5
     1     1     2     1   930     0    17     1     4    25
     8     1     1    24     4   812    10     2    20    10
     6     3     3     0     4    13   926     0     3     0
     2     7    22     3     1     0     0   973     1    19
     8     6     4     7    10    14     7     8   899    11
     7     4     0    10    26    12     1     5     6   938

Confusion matrix Test stats:
Total instances          : 10000
Overall accuracy    (OA)  : 0.94790
```

Figure-6: Confusion Matrix of Network-1 (Training and Testing)

From the confusion matrix, we can easily understand how the classifier can able to classify. Here in the confusion matrix, rows represent true class, and columns represent predictions. For training total, 60000 instances are taken, and for testing 10000.

The overall accuracy = (TP + TN) / (TP + TN + FP + FN)

The overall accuracy for training is 0.95603 or 95.603% [for 10 epochs].

The overall accuracy for testing is 0.094790 or 94.7090%

We tried to see how the number of epochs changes the loss and accuracy. From the Figure-7, we can see that training loss has decreased when the number of epochs has increased. On the contrary, the accuracy of the training has increased when the

number of epochs has increased.



Figure-7: training loss and accuracy vs. epoch (Network 1).

Sometimes it can be seen that the accuracy has increased, and the loss has also increased, which is a sign of overfitting the model. Here in Figure-7, we can see that around epoch 60 to 80,

TABLE 4.3
RELATION BETWEEN HIDDEN UNIT AND ACCURACY (TRAINING AND TESTING)

Hidden unit	Accuracy Training	Accuracy testing
32	0.9557	0.9447
64	0.97747	0.9667
96	0.98663	0.974
128	0.9906	0.9777
196	0.99482	0.9794
256	0.99637	0.983
512	0.9978	0.9852
1024	**0.99818**	**0.9854**

Summary of the relation between hidden unit and accuracy (training and testing

the network prone to become overfitted.

A modification and testing of the network by changing the hidden unit of the network. It is visible from Figure-8 that when the number of hidden units has increased, the accuracy has increased too. For the network 1, the accuracy becomes stable after 500 hidden unit. It is also clearly visible from the plot that training accuracy is higher than the testing accuracy, which feasible.

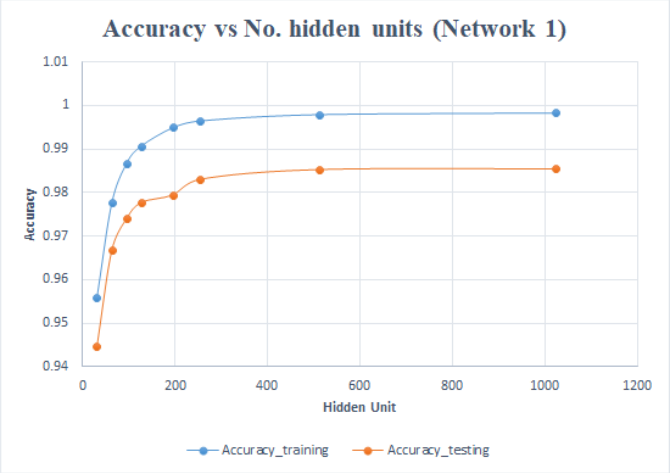


Figure-8: Accuracy vs No. hidden units (Network 1)

B. *Network II*

For the first network, the model was trained using 10 epochs initially. The Confusion Matrix for network-2 is presented in Figure-9 for both training and testing. From the confusion matrix, we can easily understand how the classifier can class the prediction. We can also see there are some miss-classify the prediction. Here in the *confusion matrix*, rows represent true class, and columns represent predictions. A total of 60000 instances are taken for training, and 10000 for testing.

The overall accuracy for training is 0.99023 or 99.023% [for 10 epochs]. The overall accuracy for testing is .98900 or 98.9%

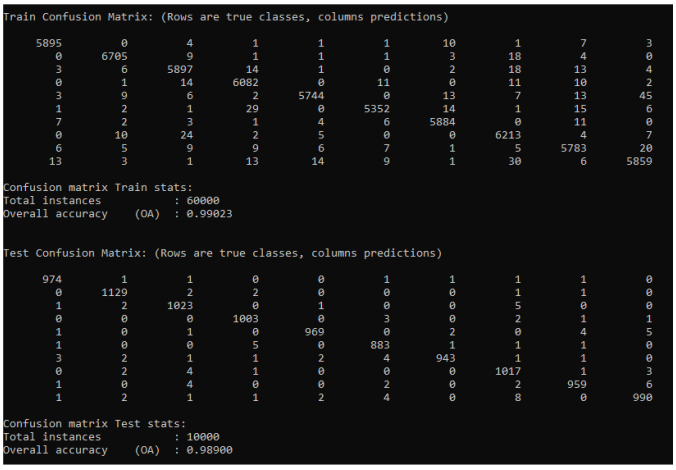


Figure-9: Confusion Matrix of Network-2 (Training and Testing)

The observation of the loss and accuracy by varying *the number of epochs.* From the Figure-10, we can see that training loss has smoothly decreased when the number of epochs has increased. On the contrary, the accuracy of the training has gradually increased when the number of epochs has increased as well.

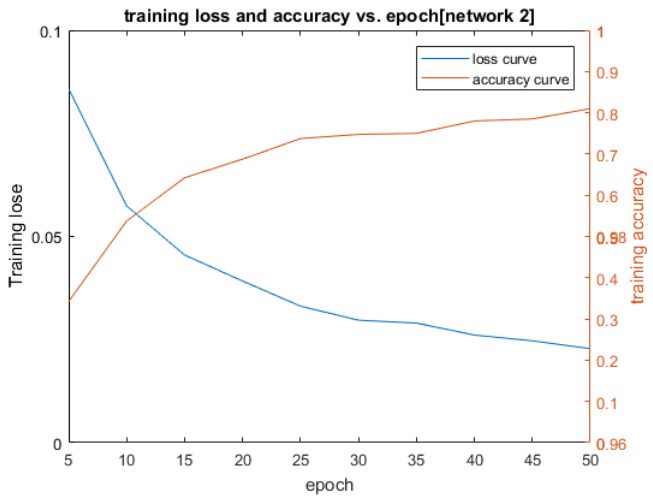


Figure-10: training loss and accuracy vs. epoch (Network 2)

C. *Network II (Modified)*

A modification was applied to network-2 for the same dataset to get better accuracy. As discussed in the approaching methods section, we have modified different parameters to get better performance. Therefore, the confusion matrix was generated. As earlier, the model has been trained with 60000 instances and got an accuracy of 0.99820 or 99.82%. the network was tested to classify the image with an accuracy of 0.9925 or 99.25%, which is pretty good results. Consideration were taken to keep in mind that the network would not become overfitted in case the model was trained for a long period. We have checked it for 10 epochs. On average, each epoch took 38 seconds, which is reasonable. The number of filters used for this case is 64. We have added a decent amount of dropout. We also tried stochastic gradient descent (SGD), which did not perform well comparing to the *'Adam' optimizer*.

```
Train Confusion Matrix: (Rows are true classes, columns predictions)

    5921        0        0        0        0        0        2        0        0        0
       0     6740        0        0        0        0        0        2        0        0
       0        2     5945        0        0        0        0       10        1        0
       0        0        5     6121        0        2        0        1        0        2
       0        2        0        0     5835        0        0        2        0        3
       0        0        0        3        0     5418        0        0        0        0
       0        5        1        1        2        9     5898        0        2        0
       0        8        1        0        0        0        0     6255        0        1
       2        1        3        2        2        4        0        0     5834        3
       4        0        0        0        9        6        0        4        1     5925

Confusion matrix Train stats:
Total instances          : 60000
Overall accuracy    (OA) : 0.99820


Test Confusion Matrix: (Rows are true classes, columns predictions)

     977        1        0        0        0        0        1        1        0        0
       0     1133        1        0        0        0        1        0        0        0
       1        1     1025        0        0        0        0        4        1        0
       0        0        1     1006        0        2        0        1        0        0
       0        0        0        0      979        0        1        0        0        2
       0        0        0        8        0      882        1        1        0        0
       2        3        0        0        2        3      946        0        2        0
       0        2        0        1        0        0        0     1023        0        2
       1        0        1        1        0        2        0        2      966        1
       0        0        0        1        6        8        1        4        1      988

Confusion matrix Test stats:
Total instances          : 10000
Overall accuracy    (OA) : 0.99250
```

Figure-11: Confusion Matrix of Modified Network-2 (Training and Testing).

VI. Conclusion

The MNIST dataset for handwritten digit recognition is analyzed in this project. For the same dataset, two different networks are used. In the first network, a 3-layer MLP with ReLU and dropout is used after each layer. The training process is fast in these network settings. We got an overall accuracy of 95% during training and 94% for testing. In network-2 a stack of CNN, RelU, and Maxpooling is used. The training process is a little bit slower, but it gives better accuracy than the Network-1. We got 99% overall accuracy for training, and 98.9% for testing. We have also modified the network-2 to get better accuracy. We have added more filter, reduced the batch size, and added a 30% dropout. This modified network-2 gave us 99.82% overall accuracy during the training and 99.25% accuracy for testing. During the training period, we always track the loss when the number of epochs has increased as it might have caused overfitting of the data. In conclusion, the Network-2 gives us better performance than Network-1 for the MNIST dataset for handwritten digit recognition.

VII. References

[1] LeCun, Yann. "The MNIST database of handwritten digits." http://yann. lecun. com/exdb/mnist/ (1998).

[2] Kussul, Ernst, and Tatiana Baidyk. "Improved method of handwritten digit recognition tested on MNIST database." Image and Vision Computing 22, no. 12 (2004): 971-981.

[3] Wei, Ran, Fugen Zhou, Bo Liu, Bin Liang, Bin Guo, and Xuanang Xu. "A CNN based volumetric imaging method with single X-ray projection." In 2017 IEEE International Conference on Imaging Systems and Techniques (IST), pp. 1-6. IEEE, 2017.

[4] Khvostikov, Alexander, Karim Aderghal, Jenny Benois-Pineau, Andrey Krylov, and Gwenaelle Catheline. "3D CNN-based classification using sMRI and MD-DTI images for Alzheimer disease studies." arXiv preprint arXiv:1801.05968 (2018).

[5] Kayalibay, Baris, Grady Jensen, and Patrick van der Smagt. "CNN-based segmentation of medical imaging data." arXiv preprint arXiv:1701.03056 (2017).

[6] "Image by PerceptiLab," *PerceptiLabs*. [Online]. Available: https://www.perceptilabs.com/. [Accessed: 13-Apr-2021].

[7] M. Isaksson, "Four Common Types of Neural Network Layers," *Medium*, 08-Jun-2020. [Online]. Available: https://towardsdatascience.com/four-common-types-of-neural-network-layers-c0d3bb2a966c. [Accessed: 13-Apr-2021].

[8] PacktPublishing, "PacktPublishing/Advanced-Deep-Learning-with-Keras," *GitHub*. [Online]. Available: https://github.com/PacktPublishing/Advanced-Deep-Learning-with-Keras. [Accessed: 13-Apr-2021].

[9] "Advanced Deep Learning with TensorFlow 2 and Keras - Second Edition," *Packt*. [Online]. Available: https://www.packtpub.com/product/advanced-deep-learning-with-tensorflow-2-and-keras-second-edition/9781838821654. [Accessed: 13-Apr-2021].

VIII. Publication Principles

The two types of contents of that are published are; 1) peer-reviewed and 2) archival. The Transactions and Journals Department publishes scholarly articles of archival value as well as tutorial expositions and critical reviews of classical subjects and topics of current interest.

Authors should consider the following points:

1) Technical papers submitted for publication must advance the state of knowledge and must cite relevant prior work.
2) The length of a submitted paper should be commensurate with the importance, or appropriate to the complexity, of the work. For example, an obvious extension of previously published work might not be appropriate for publication or might be adequately treated in just a few pages.
3) Authors must convince both peer reviewers and the editors of the scientific and technical merit of a paper; the standards of proof are higher when extraordinary or unexpected results are reported.
4) Because replication is required for scientific progress, papers submitted for publication must provide sufficient information to allow readers to perform similar experiments or calculations and use the reported results. Although not everything need be disclosed, a paper must contain new, useable, and fully described information. For example, a specimen’s chemical composition need not be reported if the main purpose of a paper is to introduce a new measurement technique. Authors should expect to be challenged by reviewers if the results are not supported by adequate data and critical details.